UNIT 1 – Django Introduction & Setup

Exhaustive Study Notes covering Core Concepts, Architecture, Setup & Command Breakdown

1. What is Django?

Django is a high-level, open-source Python web framework designed to encourage rapid web development and clean, pragmatic design. Created in 2003 by Adrian Holovaty and Simon Willison at the Lawrence Journal-World newspaper and released publicly under the BSD license in 2005, Django follows the philosophy of 'The Web framework for perfectionists with deadlines.'

Django handles much of the hassle of web development, allowing developers to focus on writing their apps without needing to reinvent the wheel. It provides an all-in-one ecosystem for building everything from simple content websites to complex enterprise platforms like Instagram, Spotify, Disqus, and Bitbucket.

■ **Key Takeaway:** Django was born in a newsroom environment, which mandated extremely fast development cycles combined with high maintainability.

2. Features of Django

Django stands out in the web development landscape due to its robust, production-grade features: • Batteries Included: Django provides out-of-the-box support for authentication, database admin interface, ORM, forms, sessions, routing, and templating. • Built-in Security: Protects automatically against Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), SQL Injection, Clickjacking, and Host Header attacks. • Extreme Scalability: Django uses a shared-nothing architecture, allowing hardware to be scaled horizontally at any tier (caching, web servers, databases). • Object-Relational Mapping (ORM): Bridge Python code directly to relational database tables without writing raw SQL queries. • Automated Admin Interface: Dynamically generates a functional back-office admin dashboard based on model definitions.

■ **Key Takeaway:** Security features in Django are enabled by default. Developers do not need to install third-party middleware for basic threat protection.

3. Advantages of Django

Choosing Django offers distinct technical advantages: 1. High Productivity & Speed to Market: Pre-built modules accelerate development speed by up to 50% compared to micro-frameworks. 2. DRY Principle (Don't Repeat Yourself): Every distinct piece of knowledge must have a single, unambiguous representation within the project. 3. Massive Ecosystem & Community: Thousands of open-source packages (django-rest-framework, django-allauth, celery, etc.). 4. Versatility: Suitable for Content Management Systems (CMS), REST APIs, Machine Learning web backends, social networks, and scientific software.

■ **Key Takeaway:** DRY reduces code duplicate bugs and simplifies project maintenance over multi-year software lifecycles.

4. Django Architecture – MVT (Model-View-Template)

Django follows a variation of the classic MVC pattern known as MVT (Model-View-Template): • Model (Data Layer): Defines database schema, data fields, validations, and relationships using Python classes inheriting from django.db.models.Model. • View (Business Logic Layer): Processes client requests, retrieves or updates models, executes business algorithms, and selects a template to render or returns JSON. • Template (Presentation Layer): Handles the user interface layout using HTML enhanced with Django Template Language (DTL) tags and variables.

Note on Controller: In Django, the framework itself acts as the Controller by parsing incoming URLs (via URLconf) and dispatching request context to the view function.

■ **Key Takeaway:** In MVC: View = User Interface, Controller = Business Logic. In Django MVT: View = Business Logic, Template = User Interface, Django Framework = Controller.

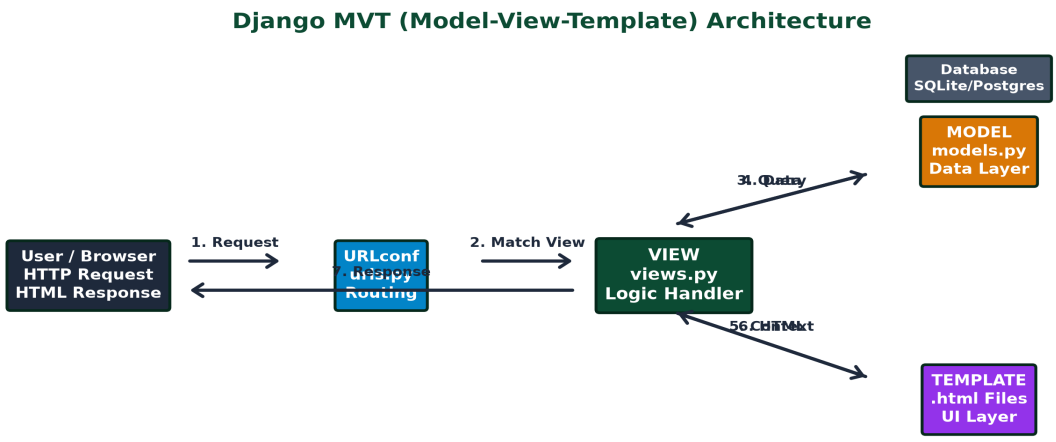

Diagram: *Django MVT Architecture showing Request-Response flow across Model, View, Template, URLconf, and Database.*

5. Django vs Flask

Feature Comparison Matrix: • Category: Django = Full-Stack Framework | Flask = Micro-Framework • Included Tools: Django = Admin, ORM, Auth, Forms built-in | Flask = Minimalist core, requires extensions (SQLAlchemy, Flask-Admin) • Project Structure: Django = Rigid, highly structured, opinionated | Flask = Flexible, developer-defined layout • Best Use Case: Django = Large enterprise apps, data portals, REST APIs | Flask = Small microservices, single-purpose APIs, light scripts

```
# Django View Example
from django.http import HttpResponse

def hello(request):
    return HttpResponse("Hello from Django!")
```

■ **Key Takeaway:** Use Django when building full-featured web portals; use Flask for lightweight microservices or small experimental prototypes.

6. Installing Django

Django requires Python 3.10+ installed. It is strongly recommended to install Django inside an isolated virtual environment (`venv`).

```
# Step 1: Create Virtual Environment
python -m venv venv

# Step 2: Activate Virtual Environment (Windows)
venv\Scripts\activate

# Step 3: Upgrade pip and Install Django
python -m pip install --upgrade pip
pip install django
```

■ **Key Takeaway:** Virtual environments prevent version conflicts between different Python projects running on the same machine.

7. Check Django Version

Verify your installation and inspect the exact active version of Django:

```
# Terminal Command
python -m django --version

# Inside Python Interactive Shell
import django
print(django.get_version())
```

8. Creating a Django Project

A Django project is a container for configuration files and modular web applications. Use `django-admin startproject` to generate the root directory.

```
django-admin startproject myproject
```

■ **Key Takeaway:** Avoid naming project folders after Python built-in modules like 'django' or 'test' to prevent namespace collision errors.

9. Creating an Application

A Django application is a Python package designed to perform a specific function (e.g., student management, payments, authentication). A project contains multiple apps.

```
# Navigate to project root containing manage.py
python manage.py startapp myapp
```

■ **Key Takeaway:** Rule of thumb: A project is a complete website; an app is a self-contained feature module within that website.

10. Project Structure

Detailed analysis of standard files generated in a Django project: • manage.py: Command-line tool to execute project management commands. • settings.py: Master configuration settings file. • urls.py: Project-level URL routing dispatcher table. • wsgi.py / asgi.py: Server deployment entry points (Sync & Async). • models.py: Data structure definitions. • views.py: Request handling functions/classes. • admin.py: Admin panel registration file.

Django Standard Project & Application Directory Hierarchy

```
myproject/                  <-- Root Project Folder
├── manage.py               <-- CLI tool for runserver, makemigrations, etc.
├── db.sqlite3              <-- Database file
├── myproject/              <-- Inner Configuration Package
│   ├── settings.py         <-- Global settings (DB, Apps, Middleware)
│   ├── urls.py             <-- Root URL Routing
│   └── wsgi.py / asgi.py   <-- Web Server Entry Points
└── myapp/                  <-- Django Application Module
    ├── admin.py            <-- Admin Interface Registration
    ├── apps.py             <-- App Configuration
    ├── models.py           <-- Database Models
    ├── views.py            <-- Business Logic Views
    └── migrations/         <-- Database Migration Files
```

Diagram: *Standard Django Project and Application Directory Hierarchy.*

11. manage.py

manage.py is a convenience wrapper created in every project root. It sets the `DJANGO_SETTINGS_MODULE` environment variable and delegates command execution to `django.core.management`.

```
# Common manage.py commands:
python manage.py runserver        # Start local dev server
python manage.py makemigrations   # Prepare DB migration scripts
python manage.py migrate          # Apply migrations to DB
python manage.py createsuperuser  # Create admin credentials
python manage.py shell            # Open interactive Django Python shell
```

12. settings.py

settings.py contains all configuration settings for your Django application: • SECRET_KEY: Used for cryptographic signing (sessions, CSRF, tokens). • DEBUG: Boolean flag (`True` in development, MUST be `False` in production). • INSTALLED_APPS: List of active Django framework apps and custom apps. • MIDDLEWARE: Sequential pipeline of request/response processors. • DATABASES: Dictionary defining database backend engines and credentials.

```
# Adding custom app to settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp', # Registered Custom App
]
```

13. urls.py

urls.py specifies the URLconf (URL configuration). Django searches through `urlpatterns` to match incoming URL paths to views.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myapp.urls')), # Include app-level URLs
]
```

14. views.py

views.py encapsulates business logic. A view takes an `HttpRequest` object as its first parameter and returns an `HttpResponse` object.

```
from django.http import HttpResponse

def home_view(request):
    return HttpResponse("<h1>Welcome to Smart Django Portal</h1>")
```

15. models.py

models.py defines database entities. Each model class maps directly to a single SQL database table.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

16. admin.py

admin.py registers models with Django's built-in administration site, allowing CRUD management from a browser dashboard.

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

17. Development Server

Django includes a lightweight, built-in development web server for rapid testing. It automatically reloads Python code upon saving changes.

```
python manage.py runserver 8000
```

■ **Key Takeaway:** DO NOT use `runserver` in production. Production deployments require WSGI/ASGI servers like Gunicorn, Uvicorn, or Nginx.

18. Request-Response Cycle

Complete step-by-step execution path of an HTTP request in Django: 1. Client browser initiates HTTP request. 2. WSGI/ASGI handler receives request and creates `HttpRequest` instance. 3. Request passes through installed `MIDDLEWARE` layers in top-to-bottom order. 4. `URLResolver` checks `ROOT_URLCONF` to match URI path to a view. 5. Target View function/class executes, querying Models or rendering Templates. 6. View returns an `HttpResponse` instance. 7. Response passes through `MIDDLEWARE` layers in bottom-to-top order. 8. WSGI server formats raw HTTP stream and delivers it to browser.

■ **Key Takeaway:** Middleware layers execute twice per request: once on the way in (Request) and once on the way out (Response).

Django Request-Response Lifecycle Flow

Diagram: *Complete Django Request-Response Lifecycle Flowchart.*